Paglot · Informe de optimización (19-10-2026)

Cómo quitar código, bajar peso y ganar velocidad sin romper nada: medido en Chrome, priorizado y con los archivos exactos. Este informe no cambia el repositorio; los scripts de medida están en `C:\Users\javie\AppData\Local\Temp\optimizacion\`.

---

1. Resumen ejecutivo

Diagnóstico en una frase. La portada descarga y ejecuta casi toda la app: 83 módulos JS (995 KB) cuando le bastan 31 (≈ 370 KB sin `en.js`), 55 hojas de estilo (752 KB) de las que usa un 15–17 %, y los 34 minijuegos con sus dos motores. Eso se nota poco en un ordenador (0,8 s) y mucho en un móvil normal: **4,1 s con la CPU 4× más lenta, 7,5 s con 4G y 21 s con 3G rápido** hasta quitar la pantalla de carga. Después de cargar, la portada de escritorio tiene el hilo principal ocupado al **72 %** (CPU 4×) solo con animaciones.

Las 10 acciones que más ganan (ganancias sobre la portada en móvil salvo que se diga otra cosa; «gz» = con gzip, que es lo que viaja con un hosting normal):

#	Acción	Ganancia estimada	Riesgo
1	**Los minijuegos y los dos motores, con `import()`**: sacar las estrellas/avance de `juegos/motor.js` a un módulo pequeño y cargar `juegos/motor.js` y `js/juego/motor.js` solo al abrir una lección (O01)	−40 módulos, −317 KB (−100 KB gz) en portada, mapa y perfil; −2 niveles de cascada	Bajo
2	**Las vistas del curso con `import()`** desde `js/app.js` (lecciones, lección, bienvenida, reto, idiomas, legal, perdida), dejando la ficha de palabra en un módulo pequeño (O02)	−12 módulos más, −170 KB; arranque estático 83 → 31 módulos y 995 → 508 KB (−49 %)	Medio
3	**No cargar `en.js` de reserva** cuando la interfaz es española o vietnamita (O03)	−137 KB (−44 KB gz), −1 nivel de cascada, −50 ms de evaluación a 4×	Bajo (comprobar.py ya garantiza las mismas claves)
4	**CSS en dos partes al publicar**: una hoja base y una por pantalla, que se pide con su vista (O04)	−400 a −480 KB de CSS bloqueante (−100 a −120 KB gz); en 3G el CSS deja de bloquear 9 s	Medio
5	**`modulepreload` generado por `publicar.py`** para los módulos del arranque (O05)	−0,6 s en 4G, −2,5 s en 3G (5 viajes de ida y vuelta menos)	Bajo
6	**`js/suave.js` en una sola pasada y con filtro rápido** (O06)	−0,5 a −1,0 s de hilo principal en el arranque del móvil (CPU 4×)	Bajo
7	**`js/paginas-movil.js`: no repartir si nada ha cambiado** y quitar los tres temporizadores fijos (O07)	−1,0 a −1,4 s de diseño forzado en los 3 primeros segundos (CPU 4×); navegación del móvil de 1,2–3 s a < 0,5 s	Medio (diapositivas: probar con `diapositivas.mjs`)
8	**Diccionario por idioma de interfaz y consejos aparte**; no traerlo «cuando esté libre» con datos móviles (O20, O21)	Español: 2,6 MB → 534 KB de núcleo (626 → 140 KB gz); −250 ms de JSON.parse a 4×	Medio
9	**Portada: banderas de la pancarta** (84 animaciones y 84 copias del SVG, 34 % del DOM) con un dibujo por bandera y pausar lo que no se ve (O30)	Hilo principal en reposo (escritorio, 4×): 72 % → ≈ 50 %; −500 nodos	Medio (arte: revisar captura)

#	Acción	Ganancia estimada	Riesgo
10	**Publicar y APK desde la misma copia**: JSON sin sangría, sin archivos de trabajo (`plantillas/` , `.md` , `.py` , `__pycache__` , `nutria.png`) y caché del service worker «primero la copia» para lo que lleva `?v=` (O40–O43)	JSON 122 → 83 MB (−32 %); APK −1,7 MB y se evita que crezca 8–15 MB al copiar `cursos/zh/plantillas` ; visitas repetidas sin red de por medio	Bajo

Objetivo realista tras las fases 1 y 2 (estimado a partir de las medidas): portada en móvil con CPU 4× de 4,1 s a ≈ 2 s; con 4G + CPU 4× de 7,5 s a ≈ 3,5 s; con 3G rápido de 21 s a ≈ 6–7 s (y ≈ 4 s si el hosting comprime). Peso de la portada: de 2.031 KB (638 KB gz) a ≈ 900 KB (≈ 280 KB gz).

Un fallo encontrado de paso: `juegos/tabla.js` no está en el mapa de importación de `index.html` (líneas 110–216): se pide sin `?v=` y un navegador o el service worker pueden servir una versión vieja tras publicar (O15).

---

2. Mediciones

2.1 Método

- **Servidor:** `http://localhost:8300` (el de `_herramientas/servir.py`): sin compresión y con `Cache-Control: no-store` . Por eso los bytes medidos son **sin comprimir**; las columnas «gz» y «br» son el tamaño que tendrían esos mismos archivos con gzip 9 / brotli 11, calculado con Node a partir de los archivos del disco.
- **Navegador:** Chrome sin ventana, perfil nuevo en cada medida, caché desactivada, por el protocolo de depuración (el patrón de `_herramientas/foto.mjs`): dominios `Network` , `Performance` , `Profiler` , `CSS` y `Tracing` .
- **Móvil:** 390×844, `mobile: true` , DPR 3. **Escritorio:** 1280×900.
- **Antes de cargar:** `paglot-sin-nube = 1` , `idiomas-ajustes = {idioma: 'es'}` (interfaz en español; en el curso de español, `en`), `idiomas-bienvenida-vista = 1` y, en las pantallas de curso, `idiomas-curso = fr` (o `es`).
- **Fin del arranque («fin»):** cuando `body` pierde la clase `arrancando` (la pantalla ya está pintada). Además: FCP, LCP (PerformanceObserver), tareas largas (> 50 ms) y su suma por encima de 50 ms («TBT»), tiempos de script/estilo/diseño de `Performance.getMetrics` , memoria JS y nodos.
- **CPU lenta:** `Emulation.setCPUThrottlingRate` 4× (móvil de gama media) y 6×. **Red:** `Network.emulateNetworkConditions` con «4G» (9 Mbps, 150 ms) y «3G rápido» (1,44 Mbps, 562 ms, el de Lighthouse).
- **Copia publicada:** `publicar.py` en `Temp\optimizacion\pub` y, en Chrome, `index.html` y la hoja unida servidos desde esa copia con `Fetch.fulfillRequest` (no se arrancó otro servidor). Esas dos respuestas no pasan por la limitación de red, así que **las medidas de la copia publicada solo valen sin limitar la red**; con red lenta se descartan.
- Scripts: `medir.mjs` (arranque), `cobertura.mjs` + `css_dom.js` (CSS y JS usados), `reposo.mjs` (coste en reposo), `perfil.mjs` (traza de tareas largas), `nav.mjs` (navegación), `suave_t.mjs` (coste de `suavizarHojas` y del diccionario), `grafo.py` (grafo de módulos), `inventario.py` , `json_stats.py` , `css_muerto.py` , `i18n_usadas.py` .

2.2 Arranque en frío, CPU normal, sin limitar la red

Pantalla	Peticiones	KB (sin comprimir)	gz	br	Fin	FCP	LCP	TBT	Tarea mayor	Memoria JS	Nodos
`#/inicio` móvil	160	2.031	638	567	810 ms	984 ms	984 ms (`h1.inicio-nombre`)	210 ms	198 ms	4,3 MB	2.081

Pantalla	Peticiones	KB (sin comprimir)	gz	br	Fin	FCP	LCP	TBT	Tarea mayor	Memoria JS	Nodos
`#/inicio` escritorio	162	2.035	640	569	805 ms	1.452 ms	1.452 ms	323 ms	305 ms	4,0 MB	2.091
`#/inicio` móvil, copia publicada (1 hoja CSS)	106	2.012	595	516	753 ms	896 ms	896 ms	227 ms	200 ms	4,2 MB	2.029
`#/lecciones` francés, móvil	156	2.443	702	616	1.160 ms	1.428 ms	1.428 ms	341 ms	186 ms	4,6 MB	4.299
`#/lecciones` francés, escritorio	156	2.445	702	616	823 ms	1.072 ms	1.204 ms	124 ms	115 ms	4,3 MB	964
`#/lecciones` español (interfaz en), móvil	155	5.080	1.314	1.059	1.033 ms	1.212 ms	1.212 ms	269 ms	174 ms	11,9 MB	4.183
`#/leccion/001` francés, móvil	158	2.459	706	619	1.504 ms	1.584 ms	1.584 ms	395 ms	248 ms	4,5 MB	504
`#/leccion/001` francés, escritorio	158	2.457	705	619	759 ms	940 ms	1.308 ms	177 ms	164 ms	4,7 MB	504
`#/leccion/001` español (interfaz en), móvil	161	5.175	1.333	1.075	1.072 ms	1.144 ms	1.144 ms	282 ms	176 ms	10,9 MB	490
`#/lecciones` español la primera vez (→ bienvenida)	188	5.646	1.400	1.134	1.061 ms	1.268 ms	2.300 ms	158 ms	151 ms	10,1 MB	600

Bytes por tipo en la portada (móvil): JS 96 peticiones / 1.184 KB (384 KB gz) · CSS 55 / 752 KB (191 KB gz) · fuentes 3 / 54 KB · imágenes 4 / 18 KB · HTML 23 KB. Los mayores: `js/i18n/es.js` 138 KB, `js/i18n/en.js` 137 KB, `css/styles.css` 83 KB, `css/precios.css` 68 KB (en la portada no se usa), `css/inicio.css` 55 KB, `js/app.js` 47 KB, `js/lessons.js` 47 KB, `css/movil.css` 44 KB, `js/vistas/lecciones.js` 44 KB, `js/juego/escenas-c.js` 40 KB (motor de juego en la portada), `js/tutora-base.js` 39 KB.

En una pantalla del curso se añade el JSON: francés 433 KB en el mapa (`indice/lecciones.json` 153 KB + `diccionario.json` 268 KB + `capitulos.json` 12 KB); español **3,2 MB** (`diccionario.json` 2.600 KB + `indice/lecciones.json` 599 KB). El diccionario se pide en el mapa «cuando el navegador está libre», con un plazo de 4 s (`js/app.js:576` y `:656`), así que acaba llegando siempre.

2.3 Con CPU lenta y con red lenta

Escenario	Fin	FCP	LCP	TBT	Script	Estilo	Diseño
Portada móvil, CPU 4×	4.099 ms	4.352 ms	4.352 ms	3.198 ms (tarea mayor 1.892 ms)	1.095 ms	1.044 ms	1.206 ms
Portada móvil, CPU 6×	6.417 ms	6.988 ms	6.988 ms	6.944 ms (tarea mayor 3.398 ms)	1.789 ms	3.312 ms	1.837 ms
Lección francés móvil, CPU 4×	3.211 ms	3.312 ms	3.312 ms	2.191 ms	1.316 ms	504 ms	1.226 ms
Lección español móvil, CPU 4×	2.955 ms	3.044 ms	3.044 ms	2.470 ms	1.292 ms	416 ms	830 ms
Portada móvil, 4G + CPU 4×	7.546 ms	6.316 ms	7.784 ms	2.306 ms			
Mapa francés, 4G + CPU 4×	11.382 ms	8.944 ms	11.892 ms	5.432 ms			

Escenario	Fin	FCP	LCP	TBT	Script	Estilo	Diseño
Lección español, 4G + CPU 4×	11.053 ms	8.508 ms	11.360 ms	4.938 ms			
Portada móvil, 3G rápido	20.943 ms	10.692 ms	21.124 ms	210 ms			

Coinciden con lo medido en el informe de compatibilidad (1,0 s normal, 10,2 s con CPU 6×, 23 s en 3G rápido). Con un hosting que comprima, los bytes por la red se dividen por 3,2 y el 3G rápido bajaría a unos 8–9 s sin tocar nada más: **la compresión del hosting es obligatoria** (ver 6.4).

2.4 La cascada de la portada en 3G rápido (cuándo empieza cada petición)

Segundo	Peticiones	KB	Qué
0	9	223	`index.html`, `es.js` (modulepreload), 3 fuentes, `fuentes.css`, `proteccion-texto.*`
1	54	793	**las 54 hojas de estilo**, todas bloqueantes
9	32	395	`app.js` y sus importaciones directas (nivel 2)
11	5	189	nivel 3: `en.js`, `srs.js`, `inicio-decorado.js`, `cursos.js`, `tutor-sprite.js`
12	6	54	nivel 3–4: `fiesta.js`, `exercises.js`, `comunes.js`, **los dos** `motor.js` …
13	2	47	`tutora-base.js`, `elenco.js`
15	7	97	`js/juego/escenas*.js`, `juegos/catalogo.js`, `juegos/lista.js`
16	32	196	**los 31 minijuegos** y `ayuda-trozos.js` (nivel 6)
20	13	35	las `tutora.js` de los cursos (pie), iconos

Dos cuellos de botella: el CSS ocupa la red los primeros 9 s y luego el grafo de módulos se descubre en **6 niveles** (cada nivel es un viaje de ida y vuelta). Los niveles 4 a 6 son solo minijuegos, que la portada no usa.

2.5 Qué parte se usa de verdad

Pantalla	CSS cargado	CSS usado (selectores que existen en el DOM)	JS ejecutado (cobertura V8, aproximada)
Portada móvil	679 KB (55 hojas)	**116 KB (17 %)**	566 de 1.163 KB (49 %)
Portada escritorio	671 KB	100 KB (15 %)	—
Lección francés móvil	679 KB	**64 KB (9 %)**	566 de 1.143 KB (50 %)

Hojas que más sobran en la portada (tamaño y % usado): `styles.css` 78 KB (12 %), `precios.css` 64 KB (4 %), `sobre.css` 34 KB (1 %), `bienvenida.css` 29 KB (3 %), `camino.css` 26 KB (6 %), `entrar.css` 25 KB (3 %), `tienda.css` 22 KB (0 %), `teatro.css` 22 KB (1 %), `empresas.css` 18 KB (1 %), `regalo.css` 18 KB (2 %), `juegos/juegos.css` 18 KB (3 %) y las 27 hojas de `juegos/css/` (≈ 95 KB, 0 %). Módulos JS con menos uso en la portada: `vistas/lecciones.js` 43 KB (8 %), `lessons.js` 46 KB (18 %), `juego/escenas-c.js` 40 KB (14 %), `vistas/bienvenida.js` 34 KB (15 %), `juego/escenas.js` 31 KB (14 %), `vistas/leccion.js` 26 KB (15 %).

2.6 Dónde se va el hilo principal (traza de la portada móvil, CPU 4×)

Tarea	Duración	Qué hay dentro
Pintar la portada (microtarea de `navegar`)	1.499 ms	921 ms de estilo y diseño **forzados** (lecturas de medidas mientras se construye)

Tarea	Duración	Qué hay dentro
`repartir()` de `js/paginas-movil.js` en un `requestAnimationFrame`	655 ms	diseño forzado: mide todas las diapositivas
Ídem (otra llamada)	415 ms	ídem
Ídem (otra llamada)	367 ms	ídem
Evaluar los módulos	524 ms	302 ms de `v8.evaluateModule` (los 83 módulos)

`repartir()` se llama en el arranque al menos seis veces: a los 400, 1.500 y 3.000 ms (`js/paginas-movil.js:234`), con `document.fonts.ready` (`:224`), con `load` y en cada `animationend` de la vista. Cada vez rehace el reparto completo aunque nada haya cambiado.

`js/suave.js`**:** en el móvil, `suavizarHojas()` recorre las 7.042 reglas de CSS con cuatro `getPropertyValue` por regla: **586 ms por pasada con CPU 4×**, y se hace dos veces (`js/suave.js:76` y `:78`, la segunda en `load`, cuando todas las hojas ya estaban).

Diccionario de español: descarga 250 ms (local), `JSON.parse` 250 ms y `new Map(...)` 84 ms con CPU 4×, más el recorrido de `elegirIdiomaContenido` (`js/lessons.js:138`).

2.7 En reposo (5 s con la pantalla ya cargada)

Pantalla	Animaciones en marcha	Hilo principal ocupado
Portada escritorio, CPU 1×	137 (88 más en pausa)	14 %
Portada escritorio, CPU 4×	137	**72 %** (estilo 1.468 ms de 5.000)
Ídem, en modo calma	7	19 %
Ídem, pausando solo `ondear` (80) y `farolillo` (30)	27	50 %
Ídem, pausando todas las animaciones CSS	0	25 % (el resto: el reloj de las tutoras)
Portada móvil, CPU 4×	6	36 % (17 % sin las animaciones CSS)
Mapa francés móvil, CPU 4×	2	22 %
Sobre, escritorio, CPU 4×	6	19 %

Todas las animaciones son de `transform` u `opacity` (bien), pero son muchas: 118 de `transform`, 17 de `opacity`, 2 de las dos. Las que más pesan: `ondear` ×80 (las tiras de las banderas, `css/inicio.css:574`), `farolillo` ×30, `titilar`, `rueda-a/b`. La pancarta de banderas son 21 banderas × 4 tiras, y **cada tira lleva una copia entera del SVG de la bandera** (`js/vistas/inicio.js:207-212`): 696 de los 2.068 nodos de la portada.

2.8 Navegación dentro de la app (móvil, ya abierta)

Ir a	CPU 1×	CPU 4×	Peticiones / KB
`#/lecciones`	130 ms	1.394 ms	1 / 268 (el diccionario)
`#/leccion/001`	115 ms	2.507 ms	2 / 15
`#/cuaderno`	77 ms	3.076 ms	1 / 11
`#/inicio`	297 ms	3.003 ms	2 / 5
`#/precios`	256 ms	1.178 ms	8 / 65

Con CPU 4× la navegación se dispara porque cada pantalla nueva vuelve a lanzar los temporizadores de `repartir()` y se encadenan tareas de 400–650 ms (2.6). Con O07 debería quedarse por debajo de 500 ms.

---

3. Peso

3.1 Inventario por carpeta y tipo (bytes reales)

Carpeta	Archivos	Tamaño	Detalle
`js/`	73	1.366 KB	3 de interfaz (`es.js` 138, `en.js` 137, `vi.js` 158 KB) = 433 KB; el resto, 933 KB
`css/`	27	628 KB	`styles.css` 82, `precios.css` 67, `inicio.css` 54, `movil.css` 43, `sobre.css` 36…
`juegos/`	63	328 KB	35 `.js` (215 KB) + 28 `.css` (113 KB)
`assets/`	18	271 KB	9 woff2 (113 KB), 6 png (141 KB, de ellos `tienda/nutria.png` **134 KB sin usar**), 2 svg (12 KB)
`cursos/`	7.243	135,6 MB	7.182 `.json` (127 MB), **12** `.pdf` **(7,6 MB), 20** `.md` **(0,5 MB), 4** `.py` **+ 4** `.pyc` **(0,4 MB)**
`index.html`	1	23 KB	55 `<link>` de CSS y 106 entradas de mapa de importación
APK de prueba	6.938 entradas	34,2 MB (122 MB sin comprimir)	`Paglot-prueba.apk` = `_apk/android/.../app-debug.apk`

Código: 31.026 líneas de JS (2.002 de ellas, comentarios de línea entera) y 13.900 de CSS; 44.927 en total.

3.2 Cursos

Curso	lessons/	indice/ (lecciones + diccionario)	Otros
es	321 · 18,7 MB	3,2 MB (598 KB + 2.599 KB)	`traducciones/` 963 archivos · 9,5 MB; `phrases` 616 KB; `juegos/` 8 · 109 KB
vi	301 · 7,0 MB	902 KB	`phrases` 75 KB
de	309 · 5,0 MB	796 KB	
id	288 · 4,7 MB	672 KB	`plantillas/` 119 KB (`.md`)
zh	299 · 4,6 MB	461 KB	`plantillas/` **15,1 MB (PDF de HSK, scripts y** `__pycache__` **)** — no se toca: lo trabaja otro agente
da	305 · 4,5 MB	822 KB	`phrases` 608 KB
it, pt, ru, pl, ro, ja, th, ko, fr, en, el, tr, nl, sv, no	2,3–4,1 MB cada uno	241–818 KB	`plantillas/` en no (305 KB), pl (90), ro (157), th (249)

3.3 Qué sobra

Qué	Dónde	Peso	Qué hacer
Material de trabajo dentro de `cursos/`	`cursos/*/plantillas/` , `cursos/th/lessons/_*.md` , `__pycache__`	8,4 MB (15,9 MB contando todo `zh/plantillas`)	**No moverlo** (otros agentes trabajan en zh, th, ro): **excluirlo al copiar** en `publicar.py` y `_apk/copiar.mjs` (O42)
Imagen sin uso	`assets/tienda/nutria.png`	134 KB	Borrar o excluir (nadie la nombra en js, css, html ni json)
Sangría de los JSON	todos los cursos (indent=1, en es/indice con CRLF)	122 → 83 MB (−39 MB, −32 %); con gzip 25,7 → 24,0 MB	Quitarla solo en la copia publicada y en la APK (O40)

Qué	Dónde	Peso	Qué hacer
CSS de otras pantallas en el arranque	`index.html:52-107`	≈ 480 KB de 752 KB	O04
Minijuegos y motores en el arranque	`js/vistas/lecciones.js:15`, `js/vistas/leccion.js:12-13`	317 KB	O01
Interfaz en inglés de reserva	`js/i18n.js:7`	137 KB	O03
Consejos de pronunciación en dos idiomas	`cursos/es/indice/diccionario.json`	1.370 KB de 2.599 (53 %), en `en` **y** `vi`	O20

4. Código: muerto, duplicado, simplificable

4.1 Dos motores de juego

	`js/juego/`	`juegos/`
Qué es	Modo juego con **guion escrito a mano** (`cursos/es/juegos/NNN.json`)	Modo juego **automático** para cualquier curso: cada bloque → un minijuego
Lecciones que lo usan	**7** (español 001–007, `cursos/es/juegos/index.json`)	todas las demás (≈ 6.200)
Código	`motor.js` 315 + `escenas.js` 761 + `escenas-b.js` 397 + `escenas-c.js` 1.063 + `efectos.js` 77 = **2.613 líneas, 105 KB**	`motor.js` 349 + `catalogo.js` 63 + `lista.js` + 31 juegos = 215 KB
Cómo entra hoy	estático desde `js/vistas/leccion.js:12` → en la portada	estático desde `js/vistas/leccion.js:13`, `lecciones.js:15` y `perfil.js:14` → en la portada

Duplicados concretos: `confeti` existe dos veces (`js/juego/efectos.js:41` y `js/fiesta.js:6`); `chispas` / `flotar` / `animar` de `efectos.js` hacen lo mismo que `js/chispa.js` y `js/fiesta.js` con otros nombres. Hay además un **ciclo** de importaciones: `juegos/motor.js:25` importa `analizarFrase` y `botonPalabra` de `js/vistas/leccion.js`, y `leccion.js:13` importa `juegos/motor.js`.

Propuesta (por orden de riesgo):

1. **O01 (ya, bajo riesgo):** mover `todasLasEstrellas`, `estrellasDe`, `avanceJuego` y sus lecturas de `localStorage` (`juegos/motor.js:47-82`, ≈ 35 líneas) a `juegos/estrellas.js` (misma carpeta, ninguna nueva). `lecciones.js` y `perfil.js` importan de ahí. Mover `analizarFrase` y `botonPalabra` (`leccion.js:98-160`) a `js/comunes.js` para romper el ciclo. En `leccion.js`, `const { pintarJuegoDeLeccion, hayJuego } = await import('../../juegos/motor.js')` al pintar, y `import('../juego/motor.js')` solo si la lección está en `cursos/es/juegos/index.json`. `hayJuego()` necesita el catálogo: se resuelve dentro del mismo `import()`, porque solo se llama al abrir la lección.
2. **O12 (más adelante, riesgo medio):** el `confeti` de `efectos.js` puede llamar al de `fiesta.js` (−15 líneas).
3. **O13 (opcional, riesgo alto, decisión tuya):** pasar los 7 guiones del español al motor automático y borrar `js/juego/` (−2.613 líneas, −105 KB). Solo si las 7 lecciones juegan igual de bien: es el contenido más cuidado, y hay que verlo antes y después.

4.2 Textos de interfaz que no usa nadie

`js/i18n/es.js` tiene 1.993 claves (`en.js` y `vi.js`, 2.000). **≈ 250 claves (12,5 %) no aparecen en el código** ni como texto ni como prefijo dinámico (se buscaron `t('…')`, `data-t`, plantillas `` `prefijo.${x}` `` y `'prefijo.' + x`; 32 prefijos dinámicos detectados). Grupos:

Grupo	Claves	Ejemplos	Origen probable
`cuenta.*`	45	`cuenta.iniciarSesion`, `cuenta.crear`, `cuenta.olvidada`	cuentas con correo y contraseña (quitadas)
`precios.*`	38	`precios.lema`, `precios.gratis.titulo`	precios antiguos (hoy `pr.*`)
`productos.*`	34	`productos.sayHelloToTheTutor`	página de productos antigua
`nube.*`	24	`nube.estado.sincronizando`, `nube.guardadoHora`	estados de la nube antigua
`comunidadExtra.*`	19	`comunidadExtra.isPaglotFree`	FAQ antigua
`form.*`	16	`form.contrasena`, `form.repiteContrasena`	contraseñas (quitadas)
`cabecera.*`	16	`cabecera.invitadoCta`, `cabecera.miCuenta`	cabecera de dos caras (obsoleta)
`repaso.*` 13 · `entrar.*` 10 · `tienda.*` 6 · `lecciones.*` 5 · `enlace.*` 5 · otros 19		`enlace.confirmada`, `entrar.olvido`	

Peso: ≈ 17–20 KB por idioma (≈ 55 KB en los tres, ≈ 750 líneas). **Cómo hacerlo sin riesgo (O10):** un script `_herramientas/claves_sin_uso.py` que haga esta misma búsqueda y liste; quitar por grupos, empezando por `cuenta`, `form`, `enlace` y `cabecera`, y pasar `comprobar.py` (check 2) y `probar_cursos.mjs`. Las que se piden con un nombre construido en tiempo de ejecución que el script no ve aparecerían en la consola como «Falta el texto» (`js/i18n.js:60`): ese aviso es la red de seguridad.

4.3 CSS muerto

`quitar_css_muerto.py` **escribe** en el archivo (no tiene modo de prueba): no se ejecutó. El análisis propio (`css_muerto.py`) marca 441 de 6.118 reglas (23 KB) cuyos selectores usan clases que no aparecen en el código, pero **la mayoría son falsos positivos**: las paletas del arte (`bz-a`, `cp-A`, `ce-D`, `a-k`, `f-l`…) se generan desde mapas de caracteres en JS. **No tocarlas.** Muertas de verdad (comprobadas):

Archivo	Clases	Reglas
`css/entrar.css`	`en-aviso`, `en-aviso-icono`, `en-cambiar`, `en-correo`, `en-enlace`, `en-o` (correo y enlaces de la cuenta antigua)	≈ 15
`css/teatro.css`	`ti-palco`, `ti-palco-der`, `ti-palco-izq`, `ti-palco-gente`	9
`css/styles.css`	`botonera-secundaria`, `mensaje-formulario`, `plegable`	9

Ganancia pequeña (≈ 4 KB); lo que de verdad pesa es cargar el CSS de todas las pantallas a la vez (O04). Para que `quitar_css_muerto.py` sea seguro, añadirle `--prueba` (que diga qué quitaría sin escribir) (O11).

4.4 Restos y exportaciones sin uso

Qué	Dónde	Líneas
Campo `tabla` (`ingles_progreso`, `espanol_progreso` …) de la nube antigua: no lo lee nadie	`js/cursos.js` (21 apariciones, p. ej. `:14`, `:36`) y `js/curso.js:43`	−22
Nombres de los cursos en `zh` y `pt` (la interfaz ya solo es es/en/vi)	`js/cursos.js` (42 líneas)	−42 (comprobar antes que `creditos.js` y la portada no los usen)
`js/cursos.js` en JSON con sangría dentro de un JS (567 líneas, 19 KB)	`js/cursos.js`	podría quedarse en ≈ 14 KB
Exportaciones que no importa nadie	`tutoraPorId` (`js/amigas.js:11`), `PRACTICAS_GRATIS_AL_DIA` y `CUADERNO_GRATIS` (`js/config.js:22,24`), `datosParaGuardar` (`js/progress.js:84`), `reemplazarDatos` (`js/progress.js:185`), `NOTAS` y `textoEspera` (`js/srs.js:10,64`), `ANIMACIONES_VALIDAS` (`js/tutora-base.js:395`)	≈ −45
Exportaciones que solo usa su propio archivo (quitar `export`)	`CLAVE_AJUSTES`, `filaAnalisis`, `CURSOS_REVISADOS`, `idDeNumero`, `revisarLeccion`, `idDeTarjeta`, `unirDatos`, `paisProbable`, `suavizarHojas`, `MOLINO`, `DATOS_DISPOSITIVO`, `estadoReto`	0 (claridad)
Comentario que remite a una herramienta que ya no existe	`juegos/lista.js:1` («Generado por `_fabrica/importmap.mjs`») y `_apk/copiar.mjs:16` (`_herramientas/generar_indices.mjs`, no existe: los índices **no se regeneran** al hacer la APK)	—
`buscar_huerfanos.py`	no encuentra archivos huérfanos (todos los `.js` y `.css` se nombran en algún sitio)	—

Total de código que se puede quitar sin decisiones: ≈ 900 líneas (750 de textos, 110 de `js/cursos.js`, 45 de exportaciones, 35 de CSS). Con O13, ≈ 3.500.

4.5 Simplificable

- `js/app.js:20-28`: 9 vistas importadas de forma estática que podrían ir por la función `vista()` que ya existe (`:78`) (O02).
- `js/vistas/bienvenida.js` importa `js/vistas/leccion.js` entero (y con él los motores de juego): 46 módulos para la bienvenida. Con O01, 18.
- `sw.js:24-29`: `guardar()` pide `c.keys()` entero en cada respuesta para recortar a 600: con 160 respuestas en el arranque son 160 listados de hasta 600 entradas. Basta recortar cada 50 respuestas o al activar (O43).
- **Siete** `fetch(..., { cache: 'no-cache' })` (`js/lessons.js:88,125,161,387,1026`, `js/juego/motor.js:37,44`) obligan a preguntar al servidor cada vez, aunque la URL ya lleva `?v=<versión>`. Con `?v=` puesto, puede quitarse `no-cache` (O16). Sin `?v=` (en desarrollo) se mantiene.

5. Velocidad

5.1 Arranque: la cascada de módulos

Grafo estático desde `js/app.js` (calculado con `grafo.py`): **83 módulos, 995 KB, 6 niveles**. La cadena más larga: `app.js → vistas/lecciones.js → juegos/motor.js → juegos/catalogo.js → juegos/pizarra.js → juegos/ayuda-trozos.js`.

Escenario	Módulos en el arranque	KB
Hoy	83	995
O01 (motores de juego con `import()`)	43	678
O01 + O02 (vistas del curso con `import()`)	31	508
O01 + O02 + O03 (sin `en.js` de reserva en es/vi)	30	≈ 371

Lo que se carga después, al abrir cada pantalla (con O01 y O02): mapa +27 módulos / 480 KB → con los motores fuera, ≈ 110 KB nuevos; lección ≈ 400 KB (aquí sí entran los juegos); bienvenida ≈ 346 KB.

O02, detalle: `app.js` usa de `vistas/leccion.js` solo `pintarLeccion`, `cerrarFicha`, `ponerDiccionarioGlobal` y `ponerDiccionario`. La ficha de palabra y el diccionario global (`leccion.js:73-96` y `:445-470`) se mueven a un módulo pequeño (`js/ficha.js`, en `js/`, sin carpeta nueva) que `app.js` importa; `pintarLeccion` va por `vista('leccion', 'pintarLeccion')`. Igual con `pintarLecciones` / `agrupar` (lecciones), `pintarReto`, `pintarElegirCurso`, `pintarLegal` / `pintarAvisoDeDatos` (este último se pinta en el arranque: dejarlo en un módulo ligero o pintarlo tras `import()`), `pintarBienvenida` / `bienvenidaVista` (la segunda es una lectura de `localStorage`: pasarla a `js/progress.js`) y `pintarPerdida`. Riesgo medio: hay que probar todas las rutas (`auditar_navegador.mjs`).

O05 `modulepreload`**:** `publicar.py` puede calcular el mismo grafo que `grafo.py` (≈ 40 líneas de Python: expresiones regulares sobre `import … from '…'`) y escribir un `<link rel="modulepreload" href="js/…?v=…">` por cada módulo del arranque, justo antes del `<script type="module">`. Así los 30 módulos se piden a la vez en lugar de en 6 oleadas. Sin O01/O02 también ayuda, pero precargaría 83 módulos: hacerlo después. CSP: no hace falta tocarla (`script-src 'self'` cubre los `modulepreload`).

`await` **de primer nivel en** `js/i18n.js:42`**.** Casi todos los módulos importan `i18n.js`, así que nada se evalúa hasta que llega el idioma. Hoy está bien resuelto porque `index.html:38-46` pide ese idioma con `modulepreload` desde el primer momento; no hay que quitarlo. Sí conviene que el preload también salga cuando el idioma es `en` (hoy se omite porque `en.js` se importa estático; con O03 dejaría de estarlo).

O03 `en.js` **de reserva.** `js/i18n.js:7` importa `en.js` siempre (137 KB) y `textoEn` (`:56-58`) lo usa si falta una clave. Pero `comprobar.py` (check 2) ya exige las mismas claves en los tres idiomas, así que con interfaz `es` o `vi` la reserva no hace falta. Cambio: cargar `en.js` con `import()` solo si la interfaz es `en`, o al primer `textoEn` sin resultado (avisando en consola), o en `patrones()` (`:110-120`, orígenes antiguos del cuaderno, que se escribieron en inglés). Riesgo bajo; probar el cuaderno con datos antiguos.

5.2 Arranque: CSS bloqueante

Las 55 hojas van en el `<head>` (`index.html:52-107`). `publicar.py` ya las une en una (`css/paglot-<versión>.css`, 736 KB): 160 → 106 peticiones y −90 ms de FCP sin limitar la red. Pero sigue siendo **todo** el CSS, y la portada usa el 17 %.

O04, CSS en dos partes, sin paso de compilación nuevo:

1. En `index.html` se quedan solo las hojas **base**: `fuentes`, `proteccion-texto`, `styles`, `cabecera`, `pie`, `escenas`, `movil`, `juego-ilustrado` (y `inicio`, que es la portada) ≈ 260 KB.
2. Cada vista declara sus hojas, por ejemplo en la tabla `GENERALES` de `js/app.js:83-100`: `precios: { …, css: ['precios'] }`, `sobre: ['sobre', 'teatro']`, lección: `['camino', 'juegos/juegos', 'juegos/css/*']`… Al navegar, `vista()` añade los `<link>` que falten y espera su `load` antes de pintar (una función de ≈ 15 líneas). Sin `@import` (prohibido en `publicar.py`).
3. `publicar.py` une **las base** en `paglot-<versión>.css` y cada grupo de pantalla en `paglot-<grupo>-<versión>.css` (la misma función `unir_css`, llamada por grupos; ≈ 30 líneas más).

Ganancia: −400 a −480 KB de CSS bloqueante (−100 a −120 KB gz); en 3G el CSS deja de ocupar los primeros 9 s y pasa a ≈ 3 s; menos reglas que recorrer en cada recálculo de estilo (hoy 4.999 reglas activas en cualquier pantalla) y en `suavizarHojas()`. Riesgo medio: una pantalla que use clases de otra hoja se vería sin estilo. Se detecta con `diapositivas.mjs` y `auditar_navegador.mjs` (28 pantallas × 4 tamaños), comparando las capturas antes/después.

Minificar CSS en `publicar.py` (O41): solo quitar comentarios y espacios al principio y al final de línea (lo seguro; no tocar espacios dentro de selectores, que cambian el significado): 741 → 643 KB, gzip 191 → 156 KB (−35 KB gz).

5.3 Arranque: fuentes

Bien resuelto: woff2 en trozos por alfabeto con `unicode-range` (`css/fuentes.css`), el latino precargado y `font-display: swap`. Se cargan 3 (54 KB). El trozo vietnamita de Be Vietnam Pro (12 KB) solo baja si hay texto vietnamita. **No hay que subsetear más:** el vietnamita necesita sus diacríticos y ya está aparte. Mejora menor: `PixelifySans.woff2` (23 KB) se precarga en `index.html:51` también en los cursos de chino, ruso, griego… donde la letra pixel se cambia (`styles.css:1702`); y no tiene `unicode-range`. Ganancia ≤ 23 KB en esos cursos (O17, P3).

5.4 Arranque en el móvil: `suave.js` y `paginas-movil.js`

- **O06 `js/suave.js`.** Quitar la segunda pasada (`:78`: los scripts de módulo se ejecutan después de cargar las hojas del `<head>`, así que en `load` no hay hojas nuevas; las que añada O04 se suavizan al añadirlas, llamando a `suavizarHojas` sobre esa hoja). Y en `suavizarEstilo` (`:38`), mirar primero `estilo.cssText.includes('steps(')` y saltar la regla si no: casi todas las 7.042 no tienen `steps()`. Ganancia: 586 ms × 2 → < 100 ms con CPU 4×. Alternativa más rápida aún (P3): generar en `publicar.py` una hoja `@media (max-width: 700px) and (prefers-reduced-motion: no-preference)` con las curvas ya cambiadas; más trabajo y más riesgo con los atajos que usan `var()`, que la versión JS ya resuelve.
- **O07 `js/paginas-movil.js`.** En `repartir()` (`:70`), guardar una firma (`innerWidth`, `innerHeight`, alto de la barra, `raiz.scrollHeight`, número de hijos) y salir si no ha cambiado. Sustituir los tres temporizadores fijos (`:234`) por un `ResizeObserver` sobre la vista (ya hay uno para la tutora, `:227-232`). Ganancia: −1,0 a −1,4 s de diseño forzado en el arranque con CPU 4× y navegación fluida. Riesgo medio: las diapositivas son una regla del dueño; comprobar con `diapositivas.mjs` en 322×645, 360×740, 390×844 y 412×915, tema claro y oscuro.

5.5 Navegación y lección

- La lección espera al diccionario completo (`js/app.js:652-654`). Con O20 espera solo al núcleo (−200 KB en francés, −2 MB en español sin comprimir).
- `cargarIndice()` (`js/lessons.js:85`) pinta el mapa con un solo archivo: bien. Pero el índice de español pesa 598 KB porque los identificadores de ejercicio llevan dentro el enunciado en cada idioma (`ejerciciosPorIdioma` 266 KB + `ejercicios` 152 KB). **No cambiar el formato de esos identificadores**: son las claves del progreso guardado. Lo que sí se puede: minificar (598 → ≈ 350 KB) y comprimir (101 KB gz).
- Al abrir una lección se piden 2–5 archivos (lección, traducciones, guion): bien. Con O16 (sin `no-cache`) la segunda visita no pregunta al servidor.

5.6 Animaciones de la portada

- **O30 Banderas.** 21 banderas × 4 tiras, cada tira con una copia del SVG (`js/vistas/inicio.js:207-213`) y su animación `ondear` con `steps(1)` y retraso por tira (`css/inicio.css:574`). Mismo dibujo con mucho menos trabajo: definir cada bandera una vez como `<symbol>` (o como imagen `data:`, que la CSP permite en `img-src`) y que cada tira lleve `<svg><use href="#bandera-xx"/></svg>`: −500 nodos aprox. Y como las 21 tiras con el mismo `--k` se mueven a la vez, se pueden **agrupar por tira** (4 capas con las 21 banderas recortadas) y animar 4 elementos en lugar de 84. El arte no cambia (mismos píxeles, mismos saltos): enseñar captura antes/después.
- `farolillo` **×30** y el resto: comprobar que el `IntersectionObserver` que ya pausa lo que no se ve (88 pausadas en escritorio) cubre también la guirnalda y la pancarta cuando se desplaza la página.
- **Reloj de las tutoras** (`js/tutora-base.js:41`): un solo reloj a 70 ms para todas (bien hecho). Cada tic redibuja la rejilla de 48×60 de cada tutora y su contorno (`outline()`, `:63-70`) aunque el fotograma sea igual al anterior. **O31:** guardar cada fotograma ya dibujado (clave: tutora + expresión + fotograma) en un `ImageBitmap` o un `canvas` fuera de pantalla y copiarlo; los píxeles son los mismos. Ganancia estimada: buena parte del 25 % de reposo que queda sin animaciones CSS en escritorio (15 tutoras en la portada). Riesgo bajo si se compara píxel a píxel.
- **CSS** `contain` **y** `content-visibility` **(O32, P3):** solo en escritorio y en pantallas largas (Sobre, Precios, FAQ, Escuelas, Empresas, capítulos del mapa) con `content-visibility: auto; contain-intrinsic-size: auto 600px`. **Nunca bajo** `html.pm-activo`: `paginas-movil.js` mide con `getBoundingClientRect()` y `content-visibility` falsearía las medidas de las diapositivas.

5.7 Memoria

4–5 MB de JS en francés y 10–12 MB en español (el `Map` del diccionario de 7.703 entradas). Nodos: 2.081 en la portada (696 son banderas), 4.299 en el mapa del móvil, ≈ 500 en la lección. No hay problema de memoria; O20 y O30 la bajan de paso (≈ −6 MB y −500 nodos).

---

6. Datos de los cursos

6.1 Lecciones: sangría y compresión

Curso	Hoy	Sin sangría	gzip hoy	gzip sin sangría
es	32,0 MB	18,7 MB	5,4 MB	5,0 MB
vi	8,0 MB	5,6 MB	1,8 MB	1,7 MB
id	6,3 MB	3,9 MB	1,2 MB	1,1 MB
da	5,9 MB	4,2 MB	1,2 MB	1,1 MB
de	5,8 MB	4,3 MB	1,4 MB	1,4 MB
fr	3,8 MB	2,8 MB	0,9 MB	0,8 MB
Todos (21)	**122,4 MB**	**83,4 MB (−32 %)**	**25,7 MB**	**24,0 MB (−7 %)**

Conclusión: con un hosting que comprime, minificar ahorra poco por la red (−7 %), pero mucho en disco, en la copia del service worker (que guarda sin comprimir) y en el `JSON.parse` (menos bytes que leer). **Hacerlo en la copia** (`publicar.py` y `copiar.mjs`), nunca en el repositorio: los JSON con sangría son los que se revisan a mano y los que editan otros agentes (O40). Comprobación de seguridad incluida: si un JSON no se lee, se copia tal cual y se avisa.

6.2 El diccionario

Variante (curso de español)	Sin comprimir	gzip
Hoy (`en` + `vi` en cada entrada)	2.599 KB	626 KB
Solo el idioma de la interfaz	1.375 KB	365 KB
+ categoría como número (12 distintas, hoy texto en dos idiomas) + sin el campo `es` repetido (7.688 de 7.703 entradas lo repiten igual que la clave)	1.207 KB	332 KB
Núcleo (sin `consejo`)	**534 KB**	**140 KB**
Los consejos aparte (`en`)	680 KB	213 KB
Por inicial (25 trozos; el mayor, la «c»)	146 KB el mayor	—

Bytes por campo hoy: `consejo` 1.370 KB, `trad` 420, `cat` 278, `pron` 88, `es` 74.

O20 (propuesta): `_herramientas/partir_diccionario.py` (Python) genera, **en la copia publicada y en la APK** (o en el repo si lo decides, salvo zh/th/ro), `indice/diccionario.<idioma>.json` con el núcleo en el idioma de la interfaz y `indice/consejos.<idioma>.json` con los consejos. `cargarDiccionario()` (`js/lessons.js:123`) pide el núcleo; la ficha de palabra pide los consejos la primera vez que se abre una (≈ 200 KB gz, una vez). Si no existen los archivos partidos, se usa el `diccionario.json` de siempre (así nada se rompe a medias). La categoría numérica obliga a tocar la ficha; puede dejarse para después (−170 KB más). En los cursos pequeños el beneficio es menor (francés 267 → 189 KB).

Partir por inicial: no compensa. La lección busca palabras de muchas iniciales a la vez (unas 15–25 peticiones por lección) y el núcleo ya cabe en 140 KB gz.

O21: en el mapa, no pedir el diccionario «cuando esté libre» si `navigator.connection?.saveData` o `effectiveType` es `2g`/`3g` (`js/app.js:576`, `:656`); se pedirá al abrir la lección.

6.3 Índices

`indice/lecciones.json`: 89–599 KB. El de español lleva los enunciados de los ejercicios dentro del identificador (ver 5.5): no tocar el formato (progreso guardado). Minificar en la copia: 598 → ≈ 350 KB. Y **falta la herramienta que los genera**: `_apk/copiar.mjs:16` llama a `_herramientas/generar_indices.mjs`, que no existe, así que la APK usa los índices que haya en cada curso (O44: recuperar el script de git o reescribirlo en Python; comprobar.py debería avisar si un índice tiene menos lecciones que `lessons/`).

6.4 Compresión en el hosting (aún sin elegir, T95)

Hosting	gzip	brotli	Nota
GitHub Pages	sí	no	No se pueden poner cabeceras de caché: los `?v=` siguen siendo necesarios
Cloudflare Pages / Netlify / Vercel	sí	sí	Cabeceras configurables (`_headers`): `Cache-Control: public, max-age=31536000, immutable` para lo que lleva `?v=`
VPS propio (nginx)	sí	con módulo	`gzip_static`/`brotli_static` con `.gz`/`.br` hechos por `publicar.py`

Con brotli: portada 2.031 KB → 567 KB; lección de español 5.175 → 1.075 KB. Elijas el que elijas, **comprobar en la primera publicación que JSON, JS y CSS salen comprimidos** (`curl -H "Accept-Encoding: br,gzip" -I`).

---

7. APK y sin conexión

7.1 Estado

- `_apk/copiar.mjs` copia `index.html`, `manifest.webmanifest`, `assets`, `css`, `cursos`, `js`, `juegos` a `_apk/www`, cambia `__VERSION__` por `Date.now()`, rehace la CSP con `publicar.py` y añade `puente-apk.js`. **No une el CSS** (55 hojas) y **copia** `cursos/` **entero**, incluidos `plantillas/`, `.md`, `.py`, `__pycache__` y PDF.
- La APK de prueba: 34,2 MB, 6.938 archivos, 122 MB sin comprimir; el diccionario de español va comprimido dentro (2,6 MB → 795 KB).
- `sw.js` no va en la APK (bien): en Capacitor el origen es `localhost` y `index.html:225` no lo registra. Sin conexión funciona porque todo está dentro.

7.2 Propuestas

ID	Qué	Ganancia	Riesgo
O42	`copiar.mjs` filtra con `cpSync(..., { filter })`: fuera `plantillas/`, `__pycache__`, `*.md`, `*.py`, `*.pyc`, `*.pdf`, `assets/tienda/nutria.png`	−8,4 MB sin comprimir hoy y se evita meter los 15 MB de `cursos/zh/plantillas` en la próxima APK (sin tocar `cursos/zh`)	Bajo
O40	La APK sale de la copia de `publicar.py` (`publicar.py <carpeta>` y `copiar.mjs` copia de ahí): CSS unido (O04), JSON sin sangría, versión y CSP ya puestas; `copiar.mjs` solo añade el puente	APK ≈ −1,7 MB; 160 → ≈ 60 lecturas de archivo al abrir; −32 % de JSON que leer	Bajo (probar la APK sin conexión, `_notas/PROBAR_LA_APK.txt`)
O45	En la APK no hace falta pedir el diccionario «cuando esté libre» ni con `no-cache`	lecturas locales más rápidas	Bajo

7.3 Service worker de la web (O43)

Hoy (`sw.js:31-41`): **primero la red** para todo; la copia solo se usa sin conexión; guarda cada respuesta y recorta a 600 entradas pidiendo la lista entera cada vez.

Propuesta, sin perder la regla de «nunca enseñar una versión vieja»:

- **Lo que lleva** `?v=<versión>` **es inmutable**: primero la copia, y si no está, la red. Al publicar cambia la versión, así que nunca sale una vieja. Ganancia: en la segunda visita la portada abre sin esperar a la red (los 106–160 archivos salen de la caché).
- `index.html` **y lo que no lleva** `?v=`: como hoy (primero la red).
- **Precarga al instalar** (`CAPARAZON`, `sw.js:7`): añadir la hoja base y los módulos del arranque, que `publicar.py` ya conoce (la misma lista que O05). Así una app instalada abre sin conexión aunque no se haya visitado cada pantalla.
- **Límite**: 600 entradas no caben en un curso de español entero (321 lecciones + 963 traducciones). Separar dos cachés: `paglot-app-<versión>` (código, sin límite, se borra al cambiar de versión) y `paglot-cursos` (lecciones, límite por número o por MB, recortando cada 50 guardados y no en cada uno).

- Riesgo: medio (un error en el service worker deja a la gente con la app rota hasta que se actualiza). Probar con «Abrir Paglot.cmd» en un nombre que no sea `localhost` o forzando el registro en una prueba, y con el modo sin conexión de Chrome.

---

8. Herramientas

8.1 Qué añadir a `publicar.py` (todo en Python, sin dependencias nuevas)

Paso	Qué hace	Líneas aprox.
CSS por grupos (O04)	`unir_css` por grupos: base + uno por pantalla; mantiene el ajuste de `url()`	+30
Minificar CSS (O41)	comentarios fuera y espacios de principio y fin de línea, nada más	+10
Minificar JSON (O40)	`json.dumps(..., ensure_ascii=False, separators=(',', ':'))` de cada `.json` de `cursos/`; si uno no se lee, se copia tal cual y se avisa	+15
Excluir material de trabajo (O42)	`shutil.copytree(..., ignore=shutil.ignore_patterns('plantillas', '__pycache__', '*.md', '*.py', '*.pyc', '*.pdf'))`	+3
`modulepreload` (O05)	grafo de importaciones estáticas desde `js/app.js`; un `<link rel="modulepreload">` por módulo	+40
Lista de precarga del service worker (O43)	escribe la misma lista en `CAPARAZON` de `sw.js`	+10
Comprimidos estáticos (si el hosting los sirve)	`.gz` (y `.br` si está el paquete `brotli`) de html, js, css, json	+15
Partir el diccionario (O20)	llama a `partir_diccionario.py` sobre la copia	+5
Informe final	tamaños antes/después por tipo, como ya hace con el CSS	+10

No unir ni minificar JavaScript. Sin un analizador de JS de verdad (que obligaría a un paso de compilación con Node), quitar comentarios de un ES module puede romper plantillas `` `…` `` y expresiones regulares. La ganancia sería pequeña: quitar comentarios de línea entera y sangría deja el JS en 1.583 → 1.330 KB, pero con gzip solo 513 → 438 KB (−75 KB en todo el código, no solo en el arranque). O01/O02/O03 ganan más (−490 KB del arranque) sin tocar el texto del código.

8.2 Qué añadir a `comprobar.py` (presupuestos que hacen fallar GitHub Actions)

Check	Regla	Umbral propuesto (hoy)
6. Mapa de importación completo	todo `.js` de `js/` y `juegos/` que se importa está en el `importmap` de `index.html`	0 que falten (hoy falta `juegos/tabla.js`)
7. Módulos del arranque	cierre estático de `js/app.js` (el grafo de `grafo.py`)	≤ 35 módulos y ≤ 420 KB tras O01–O03 (hoy 83 / 995 KB)
8. Nada de juegos en el arranque	ningún módulo de `juegos/` ni `js/juego/` en ese cierre	0 (hoy 40)
9. Peso de cada texto de interfaz	`js/i18n/*.js`	≤ 170 KB (hoy 138–158 KB)
10. CSS base	suma de las hojas enlazadas en `index.html`	≤ 300 KB tras O04 (hoy 752 KB)
11. Lección	cada `lessons/NNN.json` sin sangría	≤ 80 KB (avisar; el mayor de español, ≈ 58 KB con sangría)
12. Diccionario	núcleo por idioma	≤ 700 KB sin comprimir (avisar si se pasa)

Check	Regla	Umbral propuesto (hoy)
13. Material de trabajo	`.pdf`, `.py`, `.pyc`, `.md` dentro de `cursos/` fuera de `plantillas/`	aviso (no fallo)
14. Índice al día	`indice/lecciones.json` tiene tantas lecciones como `lessons/`	aviso

Todo esto se calcula en Python leyendo archivos, sin navegador (encaja con el trabajo `comprobar` de `.github/workflows/comprobar.yml`). Opcional: un tercer trabajo que ejecute `medir.mjs` en la portada con CPU 4× y falle si «fin» pasa de 3 s (los tiempos en GitHub Actions varían; mejor como aviso).

8.3 Herramientas existentes

- `quitar_css_muerto.py`: añadir `--prueba` (no escribir) antes de usarlo (O11).
- `buscar_huerfanos.py`: añadirle las exportaciones sin uso (el análisis de 4.4 en 25 líneas) y las claves de i18n sin uso (4.2).
- Llevar los scripts de esta medición a `_herramientas/` (no a una carpeta nueva) solo si quieres repetirla: `medir.mjs` hace el trabajo de cuatro.

---

9. Plan por fases

Cómo verificar en todas: `python _herramientas/comprobar.py` · `node _herramientas/probar_cursos.mjs 8300` (una lección de cada curso en sus dos idiomas) · `node _herramientas/auditar_navegador.mjs 8300` (28 pantallas × 4 tamaños, no debe empeorar) · `node _herramientas/diapositivas.mjs` (hoja de contactos de las diapositivas: ni zoom ni franjas) · y medir otra vez con `medir.mjs`.

Fase 1 · Rápido y seguro (1–2 días)

ID	Cambio	Archivos	Ganancia	Riesgo	Esfuerzo	Cómo comprobar
O15	`juegos/tabla.js` en el mapa de importación	`index.html` (y `publicar.py --csp`)	arregla la caché tras publicar	Bajo	5 min	comprobar.py (check 5 y el nuevo 6)
O01	Estrellas y avance a `juegos/estrellas.js`; `analizarFrase` / `botonPalabra` a `js/comunes.js`; motores con `import()`	`juegos/motor.js`, `js/vistas/lecciones.js`, `perfil.js`, `leccion.js`, `bienvenida.js`, `js/comunes.js`, `index.html` (mapa)	−40 módulos, −317 KB (−100 KB gz), −2 niveles	Bajo	3 h	probar_cursos.mjs; abrir es/001 (guion a mano) y fr/010 (automático); estrellas del mapa y del perfil iguales
O03	`en.js` solo si hace falta	`js/i18n.js`, `index.html:38-46`	−137 KB (−44 KB gz) con interfaz es/vi	Bajo	1 h	probar_cursos.mjs (los dos idiomas); cuaderno con orígenes antiguos; consola sin «Falta el texto»
O06	Una pasada y filtro rápido en `suavizarHojas`	`js/suave.js`	−0,5 a −1,0 s (CPU 4×, móvil)	Bajo	30 min	captura del móvil: las animaciones siguen suaves

ID	Cambio	Archivos	Ganancia	Riesgo	Esfuerzo	Cómo comprobar
O16	Quitar `cache: 'no-cache'` cuando hay `?v=`	`js/lessons.js`, `js/juego/motor.js`	segundas visitas sin preguntar al servidor	Bajo	20 min	publicar y abrir dos veces
O42	Excluir material de trabajo al copiar	`_apk/copiar.mjs`, `_herramientas/publicar.py`	−8,4 MB (hasta −15 MB más con zh)	Bajo	20 min	APK sin conexión; listar `www/`
O41	Minificar CSS (seguro) al publicar	`publicar.py`	−35 KB gz	Bajo	30 min	auditar_navegador.mjs sobre la copia
O40	JSON sin sangría al publicar y en la APK	`publicar.py`, `copiar.mjs`	−39 MB sin comprimir, −1,7 MB en gz/APK	Bajo	30 min	probar_cursos.mjs sobre la copia
O10	Quitar claves de interfaz sin uso (cuenta, form, enlace, cabecera…)	`js/i18n/es.js`, `en.js`, `vi.js`	−55 KB, −750 líneas	Bajo	2 h	comprobar.py; consola sin «Falta el texto» en auditar_navegador
O14	Restos: campo `tabla`, exportaciones sin uso, CSS muerto comprobado, `nutria.png`	`js/cursos.js`, `js/curso.js`, `js/config.js`, `js/srs.js`, `js/progress.js`, `js/amigas.js`, `css/entrar.css`, `css/teatro.css`, `css/styles.css`, `assets/tienda/`	−150 líneas, −134 KB	Bajo	1 h	comprobar.py; probar_nube.py (por `js/progress.js`)

Fase 2 · Arranque (3–5 días)

ID	Cambio	Archivos	Ganancia	Riesgo	Esfuerzo	Cómo comprobar
O02	Vistas del curso con `import()`; ficha y diccionario en `js/ficha.js`	`js/app.js`, `js/vistas/leccion.js`, `bienvenida.js`, `lecciones.js`, `index.html` (mapa)	arranque 43 → 31 módulos, −170 KB	Medio	1 día	auditar_navegador.mjs (todas las rutas); probar_cursos.mjs; la ficha de palabra en lección, cuaderno y frases
O04	CSS base + CSS por pantalla	`index.html`, `js/app.js` (tabla de vistas y `vista()`), `publicar.py`	−400 a −480 KB bloqueantes (−100/−120 KB gz)	Medio	1–2 días	diapositivas.mjs y auditar_navegador.mjs, capturas antes/después en los 5 tamaños y los dos temas
O05	`modulepreload` generado al publicar	`publicar.py`	−0,6 s 4G, −2,5 s 3G	Bajo	3 h	medir.mjs con 4G y 3G sobre la copia servida
O07	`repartir()` solo si cambia algo; `ResizeObserver` en lugar de tres temporizadores	`js/paginas-movil.js`	−1,0 a −1,4 s (CPU 4×); navegación < 0,5 s	Medio	4 h	diapositivas.mjs en 322×645, 360×740, 390×844, 412×915; girar el móvil; tutora que crece

ID	Cambio	Archivos	Ganancia	Riesgo	Esfuerzo	Cómo comprobar
O20	Diccionario partido: núcleo por idioma + consejos al abrir la ficha	`_herramientas/partir_diccionario.py` (nuevo), `js/lessons.js`, `js/vistas/leccion.js` (o `js/ficha.js`), `publicar.py`, `copiar.mjs`	español −2,1 MB (−486 KB gz) en la lección; −250 ms de parse	Medio	1 día	probar_cursos.mjs; ficha con consejo en es/en/vi; cuaderno (buscador)
O21	No precargar el diccionario con datos lentos	`js/app.js`	−2,6 MB en el mapa de español con 3G	Bajo	20 min	medir.mjs con 3G

Fase 3 · Fluidez y sin conexión (1 semana)

ID	Cambio	Archivos	Ganancia	Riesgo	Esfuerzo	Cómo comprobar
O30	Banderas: un dibujo por bandera y 4 capas animadas en lugar de 84	`js/vistas/inicio.js`, `css/inicio.css`	reposo 72 → ≈ 50 % (escritorio, 4×); −500 nodos	Medio (arte)	1 día	captura antes/después a 1280×900 y en las diapositivas del móvil; [V] para aprobar
O31	Fotogramas de las tutoras guardados	`js/tutora-base.js`, `js/tutor-sprite.js`	parte del 25 % de reposo restante	Bajo	1 día	comparar píxel a píxel (canvas `getImageData`)
O43	Service worker: primero la copia para `?v=`, precarga del arranque, dos cachés	`sw.js`, `publicar.py`	segundas visitas sin red; sin conexión desde la primera	Medio	1 día	modo sin conexión de Chrome; publicar dos versiones seguidas
O44	Generador de índices en Python	`_herramientas/generar_indices.py` (nuevo), `_apk/copiar.mjs`	índices siempre al día	Bajo	4 h	comprobar.py (check 14)
O45	APK desde la copia de `publicar.py`	`_apk/copiar.mjs`	APK más ligera y rápida	Bajo	2 h	`_notas/PROBAR_LA_APK.txt`
O8x	Presupuestos en `comprobar.py` (checks 6–14)	`_herramientas/comprobar.py`	que no vuelva a crecer	Bajo	3 h	GitHub Actions

Fase 4 · Opcional (decisión tuya)

ID	Cambio	Ganancia	Riesgo
O13	Pasar los 7 guiones del español al motor automático y borrar `js/juego/`	−2.613 líneas, −105 KB	Alto (las 7 primeras lecciones del curso principal)
O32	`content-visibility` en pantallas largas de escritorio	menos trabajo al desplazar	Bajo, pero nunca en el móvil
O17	`PixelifySans` sin precarga en los cursos que no la usan	≤ 23 KB	Bajo
O33	Categorías del diccionario como número	−170 KB más en español	Medio

10. Lo que no conviene tocar, y por qué

- **El dibujo de las tutoras y el arte en píxeles** (`js/tutora-base.js`, los `tutora.js` de cada curso, las paletas `bz-*`, `cp-*`, `ce-*`… de los CSS): solo se propone **guardar** lo ya dibujado (O31) y dibujar una vez lo repetido (O30), nunca cambiar un píxel. Las clases de paleta parecen CSS muerto y no lo son.
- **El estilo «a saltos» del escritorio** (`steps()`): es parte del arte; en el móvil ya lo suaviza `suave.js`. O06 lo hace más barato, no lo quita.
- **Las diapositivas del móvil y su medida** (`js/paginas-movil.js`): O07 solo evita repetir cálculos iguales. No usar `content-visibility`, `zoom` ni nada que cambie lo que mide `getBoundingClientRect()`. Nota aparte: `encoger()` (`:183-194`) aún aplica `zoom` en algunos casos, lo que va contra tu regla; es tarea del informe de arreglos, no de este.
- **Un empaquetador (esbuild, Vite, Rollup) o un minificador de JS**: ganaría quizá 50–80 KB gz más, pero rompe la regla de «sin paso de compilación», complica la CSP con hashes y el depurado. Lo que se propone (`import()`, `modulepreload`, CSS por grupos) da casi toda la ganancia sin compilar.
- **El formato de los identificadores de ejercicio** de `indice/lecciones.json`: son claves del progreso guardado (en el dispositivo y en la nube). Cambiarlos borraría progreso.
- **Las migraciones de datos antiguos** (`espanol-ajustes`, `espanol-datos`, orígenes del cuaderno en 12 idiomas en `js/i18n.js:109-130`): pesan poco y protegen datos de usuarios.
- **Los JSON del repositorio**: no quitarles la sangría en el repo (se revisan a mano y los editan otros agentes); solo en la copia publicada y en la APK. **No tocar `cursos/zh`, `cursos/th` ni `cursos/ro`**: lo que sobra en sus `plantillas/` se excluye al copiar.
- `en.js`: no borrarlo; solo dejar de cargarlo cuando no hace falta (O03).
- `entrar.js`: sigue siendo la pantalla del número de cuenta (cuenta al estilo Mullvad); lo que sobra son sus textos y estilos de correo y contraseña.
- **El `await` de `js/i18n.js:42` y el `modulepreload` del idioma en `index.html`**: están bien pensados; quitarlos obligaría a pintar dos veces.
- **Las fuentes**: ya van en trozos por alfabeto; subsetear más arriesga el vietnamita (diacríticos combinados) por unos pocos KB.
- **La CSP con hashes**: cualquier cambio en los `<script>` en línea de `index.html` obliga a `publicar.py --csp`; los `modulepreload` y `<link>` nuevos no necesitan tocarla.

Medido el 19-10-2026 sobre el árbol de trabajo (commit 3a94d1db con cambios sin subir), Chrome sin ventana en Windows 11, servidor local sin compresión. Los tiempos con CPU o red limitadas son emulados: dan el orden de magnitud y sirven para comparar antes/después; la cifra real hay que tomarla en un móvil de gama media (T63).